



FACULDADES INTEGRADAS LIBERTAS

Curso: Sistemas de Informação

Disciplina: Engenharia de Software II

Data de Entrega: 03/07/2026

Professor: Lucas Bruzzone

Integrantes (grupo de até 4): _____

Projeto Final

1. Objetivo

Aplicar, em um único projeto, as práticas vistas no bimestre: API REST em camadas, testes automatizados, mocks, análise estática, refatoração, fluxo de trabalho com Git e *code review*, containerização com Docker e *deploy* em nuvem.

O ponto de partida é um repositório **já funcional** (a API de catálogo de livros em FastAPI, com *Dockerfile*, *template* de infraestrutura e *workflow* de *deploy*). O grupo **estende** esse repositório — não constrói do zero. O que vale nota é o que for adicionado por cima da base.

2. Repositório base (entregue pronto)

- API REST em FastAPI organizada em camadas (rotas, *service*, *repository*), com o padrão Repository e modelos Pydantic.
- Dockerfile funcional.
- *infra.yml* (CloudFormation) e *deploy.yml* (GitHub Actions) para o *deploy* na AWS Academy.

3. Entregáveis obrigatórios

1. Extensão da API (Aula 12)

Adicionar pelo menos uma nova funcionalidade que contenha, obrigatoriamente:

- Uma **regra de negócio** na camada de *service* (ex.: recusar ISBN duplicado, validar ano).
- Uma **dependência externa**: ao criar um livro, consultar a API pública da Open Library pelo ISBN para validar ou completar os dados (título/autor). Endpoint gratuito e sem chave: <https://openlibrary.org/api/books?bibkeys=ISBN:<isbn>&format=json&jscmd=data>.

2. Testes automatizados (Aula 8)

- Testes **unitários** (regra de negócio do *service*) e de **integração** (endpoints via *TestClient*).
- **Cobertura mínima de 70%**, medida com *pytest-cov*.

3. Mock da dependência externa (Aula 9)

- Nos testes, a chamada à Open Library deve ser **substituída por um mock** — os testes não podem depender da rede.
- Sem o mock, a cobertura de 70% não fecha e o *pipeline* fica instável; ele é pré-condição técnica, não enfeite.

4. Pipeline de CI (Aula 9)

- *Workflow* no GitHub Actions disparado a cada *Pull Request*.
- Deve rodar *pylint* (nota mínima 7.0/10) e *pytest* com verificação de cobertura (*--cov-fail-under=70*).
- *Pull Request* com *pipeline* vermelho não pode ser *mergeado*.

5. Fluxo Git, *code review* e refatoração (Aulas 10 e 11)

- Trabalho em *branches*; nada de *commit* direto na *main*.
- Toda mudança entra por *Pull Request*.
- **Pelo menos um *code review* por integrante** (revisar o PR de outro membro, com comentário registrado).

6. Docker (Aula 13)

- A imagem deve **buildar e rodar** com a API estendida.
- Na prática, basta **atualizar as bibliotecas no `requirements.txt`** (ex.: incluir o cliente HTTP usado na chamada à Open Library) e reconstruir a imagem.

7. Deploy na AWS Academy (Aulas de *deploy*)

- **Não alterar `infra.yml` nem `deploy.yml`**: apenas **validar** que o *deploy* sobe e a API responde.
- Entregar evidência (URL ativa ou *print* da API no ar / da execução do *workflow*).

4. Distribuição de pontos

Entregável	Aula	Pontos
Extensão da API (<i>feature</i> + regra de negócio + dependência externa)	12	10
Testes unitários + integração (cobertura $\geq 70\%$)	8	30
Mock da dependência externa	9	10
Pipeline CI (pylint ≥ 8.0 + pytest + cobertura)	9	20
Fluxo Git: <i>branches</i> , PRs, <i>review</i> por integrante, PR de <i>refactor</i>	10 e 11	20
Docker buildando e rodando	13	5
<i>Deploy</i> validado na AWS Academy	—	5
Total		100

5. Regras de entrega

- Grupos de até 4 pessoas.
- Prazo: **03/07/2026**.
- **Entrega: o link do repositório no GitHub**. A avaliação considera o código, o histórico de *commits*, os *Pull Requests* e os *reviews*.
- **Apresentação de até 7 minutos** por grupo, demonstrando a API estendida, os testes e a cobertura, o *pipeline* e o *deploy* no ar.